

TPE 5

ARBRE COUVRANT MINIMAL

Algorithme de Kruskal & Structure Union-Find

VOTRE MISSION

Vous travaillez pour la **SBEE** (Société Béninoise d'Énergie Électrique). Votre mission : trouver le réseau électrique de **coût minimal** reliant tous les villages d'une commune. Pour cela, vous allez construire **pas à pas** la structure Union-Find, puis l'utiliser dans l'algorithme de Kruskal.

■ **Approche pédagogique** : on commence par une version **simple** (naïve) qui marche. Puis on découvre ses **limites**. Puis on **améliore**, une optimisation à la fois.

Repository GitHub : [graph-theory-course/session5/tpe/](https://github.com/graph-theory-course/session5/tpe/)

ARBORESCENCE DU PROJET

TPE5_Kruskal/	
include/	Fichiers d'en-tête
graph.h	Classe Graph (fourni)
union_find.h	Classe UnionFind
mst.h	Déclarations MST
src/	Code source
graph.cpp	Implémentation Graph (fourni)
union_find.cpp	À COMPLÉTER (vos TODO)
mst.cpp	À COMPLÉTER (vos TODO)
main.cpp	Programme principal
data/	Graphes de test
graph_test.txt	Petit graphe pour debug
graph_commune.txt	Graphe réaliste commune
tests/	Tests automatisés
test_mst.cpp	20 tests
Makefile	Compilation automatique

LES 6 ÉTAPES — PROGRESSION

Étape	Objectif	Points	Philosophie
1	Union-Find naïf	4	Comprendre la structure de base
2	Mesurer le problème	2	Voir POURQUOI c'est lent
3	Compression de chemin	4	Première optimisation : find rapide
4	Union par rang	3	Deuxième optimisation : arbres plats
5	Algorithme de Kruskal	7	L'algorithme principal du MST
6	Affichage & utilitaires	2	Afficher le résultat joliment
CODE	Qualité code + commentaires	+3	Noms, explications, indentation
TOTAL		25	Progression garantie !

DEADLINE : Début de la Séance 6

COMPRENDRE AVANT DE CODER

LE PROBLÈME : DÉTECTER LES CYCLES

Kruskal ajoute les arêtes une par une, de la plus légère à la plus lourde. Mais il faut **refuser** une arête si elle crée un **cycle**. Comment savoir si deux sommets sont déjà reliés ?

C'est là qu'intervient **Union-Find** : une structure qui gère des **ensembles disjoints**. Elle répond à deux questions : « *Est-ce que u et v sont dans le même groupe ?* » et « *Fusionne les groupes de u et v* ».

L'IDÉE : CHAQUE GROUPE A UN CHEF

Imaginez que chaque village a un **chef**. Quand on fusionne deux villages, un des chefs devient le chef de l'autre. Pour savoir si deux villages sont dans le même groupe, on remonte la chaîne des chefs jusqu'au chef suprême (la **racine**).

parent[] : chaque élément pointe vers son "chef"

Au début : parent[0]=0, parent[1]=1, parent[2]=2, ...

(chacun est son propre chef)

Après unite(1, 3) : parent[3] = 1 → le chef de 3 est maintenant 1

Après unite(1, 5) : parent[5] = 1 → le chef de 5 est maintenant 1

find(3) → remonte : parent[3]=1, parent[1]=1 → racine = 1

find(5) → remonte : parent[5]=1, parent[1]=1 → racine = 1

find(3) == find(5) → OUI, même groupe !

LA STRUCTURE EN C++

Classe UnionFind :

```
class UnionFind {
private:
    vector<int> parent; // parent[i] = chef de i
    vector<int> rank_; // rank_[i] = hauteur estimée (étape 4)
public:
    UnionFind(int n); // Constructeur
    int find(int x); // Trouver le chef suprême
    void unite(int x, int y); // Fusionner deux groupes
    bool connected(int x, int y); // Même groupe ?
};
```

Structure Edge (arête pondérée) :

```
struct Edge { int src, dest, weight; };
```

POURQUOI 6 ÉTAPES ET PAS 3 ?

On pourrait coder directement la version optimale. Mais vous ne comprendriez pas **pourquoi** les optimisations existent. En procédant étape par étape, vous allez :

- ① D'abord coder une version qui **marche** (étape 1)
- ② Puis **mesurer** qu'elle est lente (étape 2)
- ③ Puis **comprendre et appliquer** chaque optimisation (étapes 3-4)
- ④ Enfin utiliser le tout dans Kruskal (étapes 5-6)

ÉTAPE 1 : UNION-FIND NAÏF [4 points]

■ *Objectif : coder une version simple qui MARCHE, sans optimisation.*

FONCTION 1.1 : Constructeur UnionFind(int n)

Objectif : chaque élément est son propre chef au départ.

```
UnionFind::UnionFind(int n) : parent(n), rank_(n, 0) {  
    // TODO : Initialiser parent[i] = i pour tout i de 0 à n-1  
    // (chaque élément est son propre chef)  
}
```

FONCTION 1.2 : find(int x) — version NAÏVE

Objectif : remonter la chaîne des parents jusqu'à la racine (le chef suprême).

```
int UnionFind::find(int x) {  
    // TODO : Tant que parent[x] != x :  
    // x = parent[x] (on remonte d'un cran)  
    // TODO : Retourner x (c'est la racine)  
}
```

C'est une simple boucle while. Pas d'optimisation pour l'instant !

FONCTION 1.3 : unite(int x, int y) — version NAÏVE

Objectif : fusionner les groupes de x et y. On rattache simplement une racine sous l'autre.

```
void UnionFind::unite(int x, int y) {  
    // TODO 1 : Trouver la racine de x : rx = find(x)  
    // TODO 2 : Trouver la racine de y : ry = find(y)  
    // TODO 3 : Si rx == ry, ne rien faire (déjà dans le même groupe)  
    // TODO 4 : Sinon, rattacher : parent[ry] = rx  
    // (le chef de y devient le chef de x – choix arbitraire)  
}
```

FONCTION 1.4 : connected(int x, int y)

```
bool UnionFind::connected(int x, int y) {  
    // TODO : Retourner true si find(x) == find(y), false sinon  
}
```

■ **À ce stade : votre Union-Find fonctionne ! Testez avec `make test`. Les tests de l'étape 1 doivent passer.**

ÉTAPE 2 : MESURER LE PROBLÈME [2 points]

■ *Objectif : comprendre POURQUOI la version naïve est lente.*

Votre find() naïf remonte la chaîne parent par parent. Mais que se passe-t-il si la chaîne est **très longue** ?

```
// Imaginez cette séquence d'unions :  
unite(0, 1) → parent: 0 ← 1  
unite(1, 2) → parent: 0 ← 1 ← 2  
unite(2, 3) → parent: 0 ← 1 ← 2 ← 3  
unite(3, 4) → parent: 0 ← 1 ← 2 ← 3 ← 4  
...  
unite(998, 999) → chaîne de longueur 999 !
```

find(999) doit remonter 999 niveaux → $O(n)$ opérations !

FONCTION 2.1 : count_operations(int x)

Objectif : compter le nombre de remontées que fait find(x).

```
int UnionFind::count_operations(int x) {
// TODO : Comme find(), mais au lieu de juste retourner la racine,
// compter combien de fois on fait x = parent[x]
// et retourner ce compteur
}
```

■ Le test va créer une chaîne de 1000 éléments et vérifier que find() fait ~1000 opérations. C'est la preuve que la version naïve est $O(n)$ par appel.

■ La question : peut-on faire mieux ? Oui ! C'est l'objet des étapes 3 et 4.

ÉTAPE 3 : COMPRESSION DE CHEMIN [4 points]

■ Première optimisation : quand on cherche la racine, on en profite pour raccourcir le chemin.

L'IDÉE EN IMAGE

AVANT find(4) : APRÈS find(4) :

```
0 0
| / | \ \
1 1 2 3 4
|
2 Tous pointent directement vers 0 !
| Les prochains find() seront en O(1).
3
|
4
```

COMMENT ÇA MARCHE ?

Au lieu d'une boucle while, on utilise la **réursion**. L'astuce : après avoir trouvé la racine, on met à jour parent[x] pour pointer **directement** vers la racine.

FONCTION 3.1 : find(int x) — avec COMPRESSION

```
int UnionFind::find(int x) {
// TODO 1 : Si parent[x] == x, retourner x (c'est la racine)
// TODO 2 : Sinon, appel récursif :
// parent[x] = find(parent[x])
// ↑ C'est LA ligne magique ! Elle fait deux choses :
// a) Trouve la racine (récursivement)
// b) Met à jour parent[x] pour pointer vers la racine
// TODO 3 : Retourner parent[x]
}
```

■ Résultat : après un find(x), TOUS les noeuds sur le chemin pointent directement vers la racine. Les appels suivants sont quasi-instantanés.

■ Attention : cette version REMPLACE la version naïve de l'étape 1. Votre ancien find() est remplacé par celui-ci.

■ Le test va vérifier : après find(999) sur une chaîne de 1000, un second appel find(999) ne fait que 1 opération (au lieu de 999).

ÉTAPE 4 : UNION PAR RANG [3 points]

■ Deuxième optimisation : quand on fusionne, on choisit INTELLIGEMMENT qui devient le chef.

LE PROBLÈME DE L'ÉTAPE 1

Dans la version naïve, `parent[ry] = rx` est un choix **arbitraire**. Si on rattache toujours un grand arbre sous un petit, on crée une chaîne longue.

MAUVAIS : grand arbre sous petit BON : petit arbre sous grand

```
1 (petit) 0 (grand)
| / | \
0 (grand!) 2 3 1 (petit)
/ \
2 3 → chaîne longue ! → arbre plat !
```

LA SOLUTION : LE RANG

On maintient un **rang** pour chaque racine (estimation de la hauteur de l'arbre). Lors d'une fusion, on rattache toujours **le plus petit sous le plus grand**.

FONCTION 4.1 : `unite(int x, int y)` — avec RANG

```
void UnionFind::unite(int x, int y) {
    int rx = find(x); // racine de x
    int ry = find(y); // racine de y
    if (rx == ry) return; // déjà ensemble

    // TODO : Remplacer l'ancien "parent[ry] = rx" par :
    // Si rank_[rx] < rank_[ry] :
    // parent[rx] = ry (petit sous grand)
    // Sinon si rank_[rx] > rank_[ry] :
    // parent[ry] = rx (petit sous grand)
    // Sinon (même rang) :
    // parent[ry] = rx (choix arbitraire)
    // rank_[rx]++ (la hauteur augmente de 1)
}
```

■ **Résultat combiné (compression + rang) : chaque opération find/unite coûte $O(\alpha(n)) \approx O(1)$. C'est quasi-magique : $\alpha(n) \leq 4$ pour tout $n < 10^{80}$!**

ÉTAPE 5 : ALGORITHME DE KRUSKAL [7 points]

■ Maintenant que Union-Find est puissant, on peut coder Kruskal efficacement.

L'ALGORITHME EN 4 LIGNES

1. Extraire et trier toutes les arêtes par poids croissant
2. Créer un UnionFind(V)
3. Pour chaque arête (u, v, w) dans l'ordre :
Si !connected(u, v) : ajouter au MST + unite(u, v)
4. Retourner le MST

FONCTION 5.1 : getEdges() — Extraire les arêtes

```
vector<Edge> getEdges(const Graph& g) {  
    // TODO 1 : Pour chaque sommet u de 0 à V-1 :  
    // Pour chaque voisin (v, poids) de u :  
    // Si u < v : // ← IMPORTANT : éviter les doublons !  
    // edges.push_back({u, v, poids})  
    // TODO 2 : Retourner edges  
}
```

■■■ Pourquoi $u < v$? Le graphe est non orienté : chaque arête (u,v) apparaît deux fois dans la liste d'adjacence. En ne gardant que $u < v$, on évite les doublons.

FONCTION 5.2 : kruskal() — L'algorithme principal

```
vector<Edge> kruskal(const Graph& g) {  
    // TODO 1 : Extraire les arêtes : edges = getEdges(g)  
    // TODO 2 : Trier par poids croissant avec std::sort :  
    // sort(edges.begin(), edges.end(),  
    // [](const Edge& a, const Edge& b) {  
    //     return a.weight < b.weight;  
    // });  
    // TODO 3 : Créer un UnionFind(V)  
    // TODO 4 : Boucle sur les arêtes triées :  
    // Pour chaque Edge e :  
    // Si !uf.connected(e.src, e.dest) :  
    //     mst.push_back(e)  
    //     uf.unite(e.src, e.dest)  
    // TODO 5 : Retourner mst  
}
```

DÉROULEMENT SUR graph_test.txt (7 sommets A-G)

#	Arête	Poids	Action	Pourquoi ?
1	A — C	3	■ Accepter	A et C pas connectés
2	A — D	5	■ Accepter	D pas encore relié
3	C — E	5	■ Accepter	E pas encore relié
4	D — F	6	■ Accepter	F pas encore relié
5	A — B	7	■ Accepter	B pas encore relié
6	B — E	7	■ Refuser	B et E déjà connectés !
7	E — G	9	■ Accepter	G pas encore relié → MST complet !

MST = {(A,C,3) (A,D,5) (C,E,5) (D,F,6) (A,B,7) (E,G,9)} → poids total = 35

ÉTAPE 6 : AFFICHAGE & UTILITAIRES [2 points]

FONCTION 6.1 : mstWeight(mst)

```
int mstWeight(const vector<Edge>& mst) {  
    // TODO : Additionner e.weight pour chaque arête e du MST  
}
```

FONCTION 6.2 : printMST(mst, g)

```
void printMST(const vector<Edge>& mst, const Graph& g) {  
    // TODO : Afficher chaque arête avec g.getVertexName()  
    // Format : "A - C (poids: 3)"  
    // Puis afficher le poids total  
}
```

PIÈGES À ÉVITER

PIÈGE #1 : Oublier parent[i] = i dans le constructeur

Symptôme : find() retourne toujours 0

```
// ■ CORRECT : chaque élément est son propre chef au départ  
for (int i = 0; i < n; i++) parent[i] = i;
```

PIÈGE #2 : Oublier le cas rx == ry dans unite()

Symptôme : le MST contient des cycles

```
if (rx == ry) return; // ← Sans ça, on crée des boucles !
```

PIÈGE #3 : Écrire la compression SANS récursion

Symptôme : find() est rapide mais ne compresse pas tout le chemin

```
// ■ FAUX : ne compresse que le dernier noeud  
parent[x] = parent[parent[x]]; // compression partielle seulement  
  
// ■ CORRECT : compression complète (récursive)  
if (parent[x] != x) parent[x] = find(parent[x]);  
return parent[x];
```

PIÈGE #4 : Tri des arêtes dans le mauvais sens

Symptôme : Kruskal retourne un arbre couvrant MAXIMAL

```
// ■ Tri CROISSANT (du plus léger au plus lourd)  
sort(edges.begin(), edges.end(),  
    [](const Edge& a, const Edge& b) { return a.weight < b.weight; });
```

PIÈGE #5 : Doublons dans getEdges()

Symptôme : chaque arête apparaît 2 fois → résultats faux

```
// ■ Filtrer : ne garder que u < v  
if (u < v) edges.push_back({u, v, w});
```

TESTS, BARÈME ET RENDU

COMPILATION ET TESTS

```
Compiler : make  
Exécuter : ./bin/main  
Tests : make test && ./bin/test_mst  
Sortie : 20 tests réussis sur 20
```

BARÈME DÉTAILLÉ

Étape	Fonction(s)	Tests	Points	Ce qu'on vérifie
-------	-------------	-------	--------	------------------

1	Constructeur + find naïf + unite naïf + connected	5	4	La base fonctionne
2	count_operations	2	2	On mesure que c'est O(n)
3	find avec compression	3	4	Compression effective, find rapide
4	unite avec rang	2	3	Arbres équilibrés
5	getEdges + kruskal	5	7	MST correct sur 2 graphes
6	mstWeight + printMST	3	2	Utilitaires fonctionnels
—	Qualité du code	—	+3	Commentaires, nommage, clarté
TOT		20	25	

FORMAT DE RENDU

Fichier : `NOM_Prenom_TPE5.zip`

Contenu : `src/union_find.cpp` + `src/mst.cpp` + `capture_tests.png`

NE PAS INCLURE : fichiers .o, bin/, obj/

CHECKLIST AVANT DE RENDRE

- Les 6 étapes sont implémentées
- `make` compile sans erreurs ni warnings
- `./bin/test_mst` affiche 20/20 tests réussis
- Le code est commenté (surtout : expliquer la compression et le rang)
- Nom/Prénom en haut de `union_find.cpp` et `mst.cpp`
- Le ZIP contient `union_find.cpp` + `mst.cpp` + `capture`

CONSEILS POUR RÉUSSIR

1. **Étape 1 d'abord !** Le find naïf (boucle while) suffit pour commencer
2. **Testez après chaque étape :** `make test` à chaque fois
3. **Étape 2 = motivation :** comprendre le problème avant de l'optimiser
4. **Étape 3 = une seule ligne :** `parent[x] = find(parent[x])`
5. **Étape 5 utilise tout :** si les étapes 1-4 marchent, Kruskal est facile
6. **Dessinez le graphe** à la main pour vérifier votre MST
7. Pour le **debug** : affichez `parent[]` après chaque `unite()`
8. **N'hésitez pas à demander de l'aide !**

Bon courage !

Kruskal et Union-Find sont des structures fondamentales en informatique.
Réseaux, clustering, segmentation d'images, bioinformatique...